SPmodels: Guide to the Implemented Package

What works now, how it works, and what comes next

Olatunji Johnson

16 October 2026

Read this first

SPmodels is being developed as an independent R package for spatial modelling in research and teaching. This guide describes the package that exists now. It does not describe planned functions as though they already work.

The current package can fit **three response families** at point locations:

> Gaussian geostatistical regression by exact maximum likelihood, and binomial-logit or Poisson-log latent Gaussian geostatistical regression by a Laplace approximation with optional Monte Carlo maximum-likelihood correction. Each uses a linear mean and isotropic Matérn spatial dependence.

It can also evaluate the Matérn correlation directly, simulate fitted Gaussian, binomial or Poisson responses, calculate a Gaussian empirical residual variogram and produce family-appropriate residual diagnostics. The package now performs fixed or geometry-driven fold refitting and draw-based assessment. It produces prediction tables and point/grid or polygon target displays, and can add separable exponential temporal dependence to its Matérn spatial correlation. It does not yet supply consumer-specific cartographic styling.

This is therefore an early, usable vertical slice of the intended package—not the completed modelling collection.

What you can do now

You can currently:

1. evaluate Matérn correlations with `matern_cor()`;
2. define covariance and optimiser settings with `sp_control()`;
3. fit a Gaussian spatial regression with `sp_fit()`;
4. inspect mean coefficients, their conditional covariance and the fitted log-likelihood;

5. obtain fitted values and residuals;
6. predict at observed or new point locations;
7. request prediction standard errors for either the latent spatial surface or a future noisy observation;
8. use projected point `sf` data with explicit CRS validation;
9. simulate unconditional Gaussian responses from a fitted model;
10. calculate and plot empirical residual variograms;
11. inspect conditional and decorrelated residual diagnostics.
12. fit the corresponding independent Gaussian model with `spatial = FALSE` for limiting checks and comparisons.
13. evaluate first and second Matérn scale derivatives;
14. calculate a binomial-logit or Poisson-log latent Gaussian mode and its local curvature for supplied parameters.
15. draw latent effects conditionally on supplied parameters using a preconditioned Metropolis-adjusted Langevin algorithm.
16. fit binomial-logit and Poisson-log spatial regressions using an explicitly labelled Laplace likelihood approximation.
17. run one or several latent chains and inspect chain-specific acceptance, trace, autocorrelation, effective sample size, Monte Carlo standard error and rank-normalized split R-hat.
18. obtain binomial probability or Poisson intensity/count predictions on the response or link scale under the Laplace approximation.
19. draw predictions jointly across locations, construct projected regular grids, and calculate quantile, exceedance or weighted grouped targets.
20. refit user-supplied validation folds and assess predictive draws using errors, interval coverage and empirical CRPS.
21. refine binomial or Poisson Laplace fits with normalized importance-sampling MCML and inspect effective sample size, dominant weight and Monte Carlo error.
22. simulate fitted binomial or Poisson responses and inspect response, Pearson and signed deviance residuals.
23. construct coordinate-clustered validation folds that keep repeated locations together.
24. repeat fitted-model simulation and refitting to assess parameter bias, RMSE, empirical spread, reported uncertainty and interval coverage.
25. create stable model, prediction and validation tables without reading internal object fields.
26. build permutation or fitted-model residual-variogram envelopes and a global Monte Carlo curve test.
27. audit pairwise distance distributions with an explicit repeated-location policy.
28. select a UTM CRS with explicit zone, hemisphere, equator and boundary rules.
29. aggregate polygon-cell predictions using exact fractional intersection areas and optional cell density.
30. join unrounded point/grid or polygon target summaries to `sf` geometry and display a selected numerical summary.
31. assess predictive interval calibration across nominal levels with Wilson bounds, and

display validation or simulation results from stored values.

32. compare matched baseline and updated prediction scenarios while preserving fitted parameters, row order, locations, geometry and untouched columns.
33. format stable tables and assemble model, prediction and validation report sections without replacing their retained unrounded values.
34. fit additive Gaussian random intercepts for one or more grouping variables, estimate their variances, and predict or simulate at known levels or under an explicit marginal policy for unseen levels.
35. repeat MCML fits with independent importance draws and inspect between-replicate parameter movement without conflating it with sampling uncertainty.
36. increase MCML sample size geometrically until declared relative-error and effective-sample targets are met or an auditable maximum is reached.
37. represent a decay-adjusted intervention history and carry its prevalence multiplier through binomial fitting, prediction, simulation, MCML and validation.
38. estimate constrained intervention impact/decay parameters by a labelled Laplace criterion and display their effect curve with delta-method bounds.
39. declare survey time with `sp_temporal()`, fit separable space-time dependence, forecast at new times, and estimate the temporal scale in a Gaussian, binomial or Poisson model.
40. run separate simulation/refit recovery experiments across multiple named fitted scenarios while retaining every scenario result and failure.
41. request support-respecting local parameter intervals with `confint()`.
42. evaluate held-out Gaussian densities or integrated binomial/Poisson predictive log scores with `sp_predictive_log_score()`.
43. exclude a declared distance around held-out spatial folds with `sp_buffered_folds()`.
44. fit additive latent random intercepts in Gaussian, binomial-logit or Poisson-log spatial models and predict conditionally at known levels or marginally at declared unseen levels.
45. construct regular spatial-block folds with declared widths and origin, keeping every occupied block intact under systematic or seeded allocation.
46. construct environmental-space folds from declared numeric covariates, optionally standardized, while keeping repeated locations together.
47. dispatch named fitted-scenario recovery experiments across optional non-Windows workers without changing preassigned scenario random streams.
48. dispatch individual recovery replicates across optional non-Windows workers with pre-generated responses and explicit MCML seeds.
49. escalate complete simulation-recovery experiments to a declared bias-MCSE target or exact replicate cap with every attempt retained.
50. calculate empirical Pearson, deviance or seeded randomized-quantile residual variograms for fitted binomial and Poisson models.
51. build permutation envelopes under an explicit exchangeability null, or fitted-model parametric envelopes that simulate and refit each discrete- family replicate, with a global curve test and retained convergence audit.
52. add one or more grouped random slopes, including a correlated random intercept and slope, with either fixed covariance or opt-in constrained estimation throughout

fitting, prediction, simulation, MCML, diagnostics and refitting workflows.

53. use a normalized, balanced multiple-scale MCML proposal and quantify parameter-level Monte Carlo covariance across independent fits, separately from fitted-Hessian sampling covariance.
54. compute exact pointwise predictions in bounded-memory chunks while retaining ordinary means, marginal standard errors, row order and grouping policies.
55. optionally adapt an MCML proposal with an independent discarded pilot, regularized importance-weighted moments and a separately diagnosed main sample.
56. draw from a bounded-memory nearest-predecessor approximation to the joint predictive distribution, with an exact all-predecessor boundary and retained order/neighborhood audit data.
57. stream grouped weighted targets from that approximation while releasing point-draw vectors after their final neighbor-graph use.
58. reconstruct its implied covariance analytically on manageable validation subsets and compare it with the exact fitted covariance without Monte Carlo error.
59. fit an explicit Gaussian nearest-predecessor training likelihood from bounded local covariance blocks, with a profiled mean, retained graph and exact all-predecessor boundary.

Public interface implemented now

Operation	Public call	Current meaning
Matérn correlation	`matern_cor()`	Correlation for non-negative distances
Space-time correlation	`spacetime_cor()`	Separable Matérn spatial and exponential temporal correlation
Temporal specification	`sp_temporal()`	Survey-time column, temporal scale, units and fixed/estimated choice for every supported family
Fit controls	`sp_control()`	Smoothness, spatial/group variance starts or fixed values, units and optimiser controls
Model fitting	`sp_fit()`	Gaussian exact ML or binomial/Poisson Laplace geostatistical regression
Scalable Gaussian fitting	`sp_fit_neighbor()`	Explicit order-dependent nearest-predecessor ML approximation; exact likelihood boundary with all predecessors

Operation	Public call	Current meaning
Grouped slope specification	`sp_random_slope()`	Fixed or constrained-estimated positive-definite slope or correlated intercept-slope covariance by group
MCML correction	`sp_mcml()`	Importance-sampling marginal-likelihood refinement of a non-Gaussian fit
MCML controls	`sp_mcml_control()`	Main/pilot draw counts, proposal scales, optional regularized adaptation, optimizer and ESS warning threshold
MCML replication	`sp_mcml_replicates()`	Independent fits, optimizer/natural-scale Monte Carlo covariance and retained diagnostics
MCML precision	`sp_mcml_precision()`	Declared MCSE/ESS targets, proposal refresh and capped sample-size escalation
Intervention history	`sp_intervention()`	Fixed or fitted immediate reduction/decay, optional alpha penalty, unit/time history and same-time rule
Intervention multiplier	`sp_intervention_multiplier()`	Auditable row-level prevalence multipliers for survey or prediction data
Intervention curve	`sp_intervention_curve()`	Remaining reduction, multiplier and fitted delta-method pointwise bounds
Mean coefficients	`coef(fit)`	Family-appropriate fitted regression estimates
Coefficient covariance	`vcov(fit)`	Gaussian conditional covariance or local Laplace-Hessian approximation
Parameter intervals	`confint(fit)`	Identity, log or logit local Wald intervals according to parameter support

Operation	Public call	Current meaning
Model summary	`summary(fit)`	Coefficients, covariance parameters and log-likelihood
Model table	`sp_model_table()`	Stable mean/covariance estimate schema for applications and reports
Spatial prediction	`predict(fit, ...)`	Gaussian kriging or Laplace-integrated binomial/Poisson point prediction
Chunked prediction	`sp_predict_chunked()`	Exact pointwise prediction with bounded cross-covariance memory
Joint prediction	`sp_predict_draws()`	Spatially dependent link or response draws across points
Scalable joint draws	`sp_predict_draws_neighbor()`	Order-dependent nearest-predecessor approximation without a full grid covariance; exact with all predecessors; optional fitted-parameter layer
Streamed scalable targets	`sp_predict_target_neighbor()`	Immediate grouped aggregation with sparse weights and recorded peak active draw storage
Neighbor covariance audit	`sp_neighbor_covariance()`	Full diagnostic recursion and optional exact covariance-error metrics on manageable subsets
Prediction grid	`sp_grid()`	Stable regular point grid inside a projected boundary
Polygon allocation	`sp_polygon_weights()`	Assign prediction rows and optional raw weights to polygon regions
Draw-based target	`sp_predict_target()`	Quantiles, exceedance probabilities and grouped weighted means
Prediction table	`sp_prediction_table()`	Stable mean, SD, quantile and exceedance schema from joint draws

Operation	Public call	Current meaning
Prediction scenario	`sp_prediction_update()`	Matched prediction changes after audited predictor or formula-offset updates
Cross-validation	`sp_cross_validate()`	Held-out refitting, predictions and fold/overall scores
Predictive log score	`sp_predictive_log_score()`	Gaussian density or response mass integrated over a Gaussian link prediction
Spatial folds	`sp_spatial_folds()`	Coordinate-clustered folds with repeated-location protection
Buffered folds	`sp_buffered_folds()`	Geometry-driven test folds with fold-specific near-test training exclusion
Regular block folds	`sp_block_folds()`	Whole-block deterministic or seeded allocation with retained grid metadata
Environmental folds	`sp_environmental_folds()`	Unique-location clustering in declared covariates with retained scaling and centres
Group-constrained folds	`sp_group_folds()`	Whole-unit folds with greedy row balancing and retained allocation metadata
Draw assessment	`sp_simulation_assess()`	Recovery error, coverage, interval width and empirical CRPS
Calibration	`sp_calibration()`	Nominal versus empirical predictive interval coverage with Wilson bounds
Validation table	`sp_validation_table()`	Common long-form metrics for validation and simulation results
Formatted table	`sp_format_table()`	Presentation-only fixed-decimal view retaining the exact source table
Analysis report	`sp_report()`	Model, optional prediction and validation sections from stable public tables

Operation	Public call	Current meaning
Parameter recovery	`sp_simulation_recovery()`	Repeated response simulation, refitting, bias, RMSE, spread and coverage
Recovery grid	`sp_simulation_grid()`	Cartesian design over recoverable generative parameters with rebuilt covariance state
Recovery scenarios	`sp_simulation_scenarios()`	Auditable recovery experiments across named fitted models and replicate counts
Recovery precision	`sp_simulation_precision()`	Bias-MCSE target, minimum successes, geometric escalation, exact cap and optional replicate workers
Fitted values	`fitted(fit)`	Conditional response means at observed locations
Residuals	`residuals(fit)`	Observed response minus fitted conditional prediction
Log-likelihood	`logLik(fit)`	Exact Gaussian or approximated non-Gaussian marginal log-likelihood
Model simulation	`simulate(fit)`	Gaussian, binomial or Poisson unconditional response simulations at fitted sites
Empirical variogram	`sp_variogram(fit)`	Gaussian fitted curve or empirical-only discrete-family residual semivariances
Variogram envelope	`sp_variogram_envelope()`	Any-family permutation or Gaussian fitted-model pointwise envelope and global test
Residual diagnostics	`sp_diagnostics(fit)`	Gaussian decorrelated or non-Gaussian response/Pearson/deviance/randomized-quantile residuals
Distance audit	`sp_distance_summary()`	Pair counts, duplicate policy and type-8 distance summaries

Operation	Public call	Current meaning
UTM CRS choice	`sp_utm_crs()`	Audited WGS84 UTM zone/hemisphere selection without hidden transformation
Independent model	`sp_fit(..., spatial = FALSE)`	Gaussian maximum-likelihood linear model without spatial dependence
Matérn derivatives	`matern_scale_derivative()`	First or second scale derivative
Latent mode	`sp_latent_mode()`	Deterministic mode for supplied binomial/Poisson model components
Latent sampler	`sp_latent_sample()`	Conditional MALA draws for supplied parameters
Sampler controls	`sp_mcmc_control()`	Chains, dispersed starts, iterations, burn-in, thinning and step size
Sampler diagnostics	`sp_sampler_diagnostics()`	Chain acceptance, trace/ACF, pooled ESS/MCSE and rank-normalized split R-hat
Fractional polygon weights	`sp_polygon_area_weights()`	Exact projected cell-region areas with optional density
Spatial target result	`sp_target_geometry()`	Exact identifier/row join of target summaries to `sf` geometry

The scientific model in plain language

Suppose a continuous outcome is measured at several locations. Examples might include temperature, pollution concentration or a transformed concentration of a chemical. Nearby outcomes can be similar even after known predictors have been included. An ordinary linear model does not represent that remaining spatial similarity.

SPmodels divides each observation into four parts:

1. a regression mean, such as an intercept plus an elevation effect;
2. a known offset, if the user supplied one;
3. a spatial effect shared more strongly by nearby sites;
4. independent noise specific to the observation.

For site i, write this as

$$\boxed{Y_i = x_i^\mathsf{T}\beta + o_i + S_i + \varepsilon_i}$$

where:

- Y_i is the observed continuous response;
- x_i is the row of predictors created from the R formula;
- β contains the regression coefficients;
- o_i is a known offset and is zero when no offset is supplied;
- S_i is the latent spatial effect;
- ε_i is independent observation-level noise.

The model assumes

$$S \sim N(0,\, \sigma_s^2 R), \qquad \varepsilon \sim N(0,\, \tau^2 I), \qquad S \text{ is independent of } \varepsilon.$$

The two variance parameters have different meanings:

- σ_s^2, named `spatial_variance`, measures variation in the smooth latent spatial surface;
- τ^2, named `nugget_variance`, measures independent noise and very small-scale variation not represented by the spatial surface.

Independent comparison

Setting `spatial = FALSE` removes S_i from the model. SPmodels then fits

$$Y_i = x_i^\mathsf{T}\beta + o_i + \varepsilon_i, \qquad \varepsilon_i \sim N(0, \tau^2),$$

using ordinary least-squares coefficients and the maximum-likelihood residual variance $n^{-1}\sum_i e_i^2$. Coordinates and distance units are not needed. This provides a transparent comparison for deciding whether modelling spatial dependence materially changes the analysis.

How distance creates spatial correlation

The current implementation accepts either two numeric coordinate columns or a projected point `sf` object. Both axes must use the same Euclidean unit. For ordinary data frames that unit must be stated explicitly; for `sf` data it is read from the projected CRS.

For locations $s_i = (s_{i1}, s_{i2})$ and $s_j = (s_{j1}, s_{j2})$, SPmodels calculates

$$d_{ij} = \sqrt{(s_{i1} - s_{j1})^2 + (s_{i2} - s_{j2})^2}.$$

It then converts distance into correlation using the Matérn function

$$\boxed{R(d) = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{d}{\rho}\right)^{\nu} K_{\nu}\left(\frac{d}{\rho}\right)} \qquad (d > 0),$$

with $R(0) = 1$. The symbols mean:

- d: distance between two sites;
- ρ: `scale`, expressed in the coordinate unit;
- ν: `smoothness`, a positive dimensionless number;
- K_{ν}: a standard modified Bessel function;
- Γ: the gamma function.

The interpretation is more important than the special functions:

- correlation equals one at zero distance;
- correlation declines towards zero as distance increases;
- a larger scale ρ makes correlation decline more slowly;
- smoothness ν controls how smooth the latent spatial surface is.

The default is $\nu = 0.5$. At this value the equation becomes the familiar exponential correlation

$$\boxed{R(d) = \exp(-d/\rho).}$$

This identity is also used as an independent check of the implementation.

For all observed locations, the response covariance matrix is

$$\boxed{V = \sigma_s^2 R + \tau^2 I.}$$

The spatial part supplies correlated variation. The nugget is added only to the diagonal because its errors are independent between observations.

Optional residual dependence over time

For repeated surveys, add `time = sp_temporal(~ year, scale = 2, time_units = "years")` to `sp_fit()`. For observations at locations s_i and times t_i, the latent correlation becomes

$$R_{ST}(i, j) = R_M(d_{ij}; \rho, \nu) \exp\{-|t_i - t_j|/\lambda\},$$

where $\lambda > 0$ is the temporal scale. Dependence must therefore be close in both space and time to remain strong. `scale` is fixed by default. Setting `estimate = TRUE` inside

`sp_temporal()` estimates it jointly with the Gaussian spatial covariance or within the non-Gaussian Laplace criterion. MCML subsequently re-estimates the same positive log-scale parameter. Prediction data must carry the same declared time variable. Point and joint prediction, simulation, fitted variograms, cross-validation and reporting all retain the time structure.

Data required by `sp_fit()`

The current fitting path requires:

- an ordinary R `data.frame` or a projected point `sf` object;
- a Gaussian numeric response, binomial response encoding, or non-negative integer Poisson response;
- a model formula such as `temperature ~ elevation + rainfall`;
- exactly two finite numeric coordinate columns;
- at least two distinct locations;
- explicit common distance units;
- a full-rank model matrix;
- Gaussian identity, binomial logit or Poisson log family.

For an ordinary data frame, coordinates are supplied with a one-sided formula:

```
coords = ~ east_km + north_km
```

For `sf` input, omit `coords`. SPmodels requires non-empty two-dimensional point geometry with a known projected CRS. Longitude/latitude data are rejected, and prediction data must use the fitted CRS. The package does not guess a projection or silently transform prediction data.

By default, missing values cause an explicit error through `na.fail`. This is intentional: silently removing spatial observations can break alignment between responses, predictors and coordinates.

A complete reproducible example

The following example creates a small artificial data set. It is intended to show the interface, not to support a scientific conclusion.

```
library(SPmodels)

set.seed(20261013)
```

```r
dat <- expand.grid(
  east_km = 0:4,
  north_km = 0:3
)

dat$elevation <- 100 + 15 * dat$east_km + 8 * dat$north_km
dat$outcome <-
  4 +
  0.03 * dat$elevation +
  0.8 * sin(dat$east_km / 2) +
  rnorm(nrow(dat), sd = 0.35)
```

Fit the model:

```r
fit <- sp_fit(
  outcome ~ elevation,
  data = dat,
  coords = ~ east_km + north_km,
  control = sp_control(distance_units = "km")
)
```

Inspect it:

```r
print(fit)
summary(fit)
coef(fit)
vcov(fit)
logLik(fit)
```

Create two prediction locations. They must contain every predictor used by the mean formula and both coordinate columns.

```r
new_sites <- data.frame(
  east_km = c(0.5, 2.5),
  north_km = c(0.5, 1.5),
  elevation = c(111.5, 149.5)
)

latent_prediction <- predict(
  fit,
  newdata = new_sites,
  se.fit = TRUE,
  target = "latent"
)
```

```r
observation_prediction <- predict(
  fit,
  newdata = new_sites,
  se.fit = TRUE,
  target = "observation"
)
```

Both calls return the same conditional mean. The observation standard error is larger because it includes the nugget variance of a future measurement.

What `sp_control()` controls

The default call is

```r
sp_control(distance_units = "km")
```

Its arguments are:

Argument	Default	Meaning now
`smoothness`	`0.5`	Fixed Matérn smoothness ν
`spatial_variance`	automatic	Starting value for σ_s^2, or fixed value
`scale`	automatic	Starting value for ρ, or fixed value
`nugget_variance`	automatic	Starting value for τ^2, or fixed value
`estimate`	`TRUE`	Estimate all three covariance parameters?
`distance_units`	`NULL`	Required description such as `"m"` or `"km"`
`maxit`	`500`	Maximum optimiser iterations
`reltol`	`1e-8`	Optimisation tolerance

Matérn smoothness is fixed in this implementation. When `estimate = TRUE`, the other three covariance parameters are estimated jointly. Values supplied for them are starting values.

When `estimate = FALSE`, all three must be supplied and are treated as known:

```
fixed_control <- sp_control(
  smoothness = 0.5,
  spatial_variance = 1.2,
  scale = 3,
  nugget_variance = 0.2,
  estimate = FALSE,
  distance_units = "km"
)
```

Fixed values are useful for analytic checks and carefully specified teaching examples. They should not be chosen merely to force a desired conclusion.

How the model is fitted

Stack the responses into y, offsets into o, and formula-derived predictors into the design matrix X. The model implies

$$Y \sim N(X\beta + o, V)\,.$$

For fixed covariance parameters, the Gaussian log-likelihood is

$$\ell(\beta, V) = -\frac{1}{2}\left[n\log(2\pi) + \log|V| + r^{\mathsf{T}}V^{-1}r\right],$$

where

$$r = y - o - X\beta.$$

The coefficient estimate that maximises this likelihood for a given covariance matrix is

$$\boxed{\hat{\beta} = \left(X^{\mathsf{T}}V^{-1}X\right)^{-1}X^{\mathsf{T}}V^{-1}(y-o).}$$

This is generalised least squares. It differs from ordinary least squares by allowing observations to have spatial covariance.

SPmodels substitutes this coefficient estimate into the likelihood and optimises the remaining covariance parameters. It optimises their logarithms, which guarantees positive parameter values. This is ordinary maximum likelihood, not restricted maximum likelihood and not Bayesian inference.

The default automatic starting values are:

- scale: the median positive pairwise distance;
- total starting variance: based on ordinary least-squares residuals;

- 80% of that variance assigned initially to the spatial component;
- 20% assigned initially to the nugget.

The implementation uses Cholesky factorisation and triangular solves. It does not form a matrix inverse during fitting and does not silently add numerical jitter. Dense factorisation needs roughly cubic computation time and quadratic memory as the number of observations grows.

What coefficient uncertainty means

Conditional on the fitted covariance parameters,

$$\mathrm{Var}\big(\hat{\beta} \mid \widehat{V}\big) = \left(X^{\mathsf{T}}\widehat{V}^{-1}X\right)^{-1}.$$

This is what `vcov(fit)` returns and what `summary(fit)` uses for coefficient standard errors. It treats the fitted covariance parameters as fixed. It does not include uncertainty caused by estimating spatial variance, scale or nugget variance. The summary therefore does not yet provide complete small-sample inference.

How prediction works

Gaussian prediction

At a new location, SPmodels combines:

1. the regression mean at that location;
2. information borrowed from residuals at observed locations;
3. the spatial correlations between the new and observed locations.

Let c_* contain covariances between the latent spatial effect at the new site and responses at all observed sites. The predicted value is

$$\boxed{\widehat{Y}_* = x_*^{\mathsf{T}}\hat{\beta} + o_* + c_*^{\mathsf{T}}V^{-1}(y - o - X\hat{\beta}).}$$

The final term is the spatial adjustment. Nearby residuals generally receive more influence because their covariance with the target is larger.

Latent target

`target = "latent"` describes uncertainty about the smooth underlying spatial surface. Its prediction variance is

$$v_{*,\text{latent}} = \sigma_s^2 - c_*^\mathsf{T} V^{-1} c_* + q_*^\mathsf{T} \left(X^\mathsf{T} V^{-1} X \right)^{-1} q_*,$$

where

$$q_* = x_* - X^\mathsf{T} V^{-1} c_*.$$

The second term is the reduction in uncertainty obtained from observed sites. The final term allows for estimation of the regression mean.

Future-observation target

`target = "observation"` predicts a new noisy measurement. Its variance is

$$\boxed{v_{*,\text{observation}} = v_{*,\text{latent}} + \tau^2.}$$

The prediction mean is the same for both targets. Their standard errors differ. Neither currently includes uncertainty in the fitted covariance parameters.

Binomial and Poisson prediction

For a non-Gaussian fit, the Laplace approximation gives an approximately normal predictive linear predictor,

$$\eta_* \mid y \dot{\sim} N(m_*, v_*).$$

`type = "link"` returns m_* and $\sqrt{v_*}$. On the response scale, SPmodels integrates over this distribution. For Poisson data,

$$E(\mu_* \mid y) = \exp(m_* + v_*/2),$$

which is not simply $\exp(m_*)$. For binomial data, the corresponding logistic-normal mean has no elementary formula, so it is evaluated by numerical integration. `target = "latent"` describes the smooth intensity or probability; `target = "observation"` also includes the new independent latent effect and response sampling variation in the reported standard error. Regression- and covariance-parameter uncertainty is not propagated.

Bounded-covariance-memory Gaussian fitting

`sp_fit_neighbor()` is a separate opt-in Gaussian fitter. It orders the observations, conditions each observation on at most the requested number of nearest earlier observations, and profiles the regression coefficients from the resulting weighted transformed regression. The returned object records the order, graph, local conditional coefficients and variances. Its ordinary `coef()`, `vcov()`, `logLik()`, `fitted()`, `residuals()`, `summary()` and pointwise `predict()` methods do not require a dense training covariance.

Keeping all predecessors reproduces the dense Gaussian log likelihood, regression estimates and conditional coefficient covariance to numerical precision. Smaller budgets are approximations whose result can depend on the input, deterministic maximin or explicitly supplied order. Local prediction conditions a new point on its nearest training observations and returns no joint covariance.

This first implementation supports only the stationary Gaussian Matérn plus nugget model. It deliberately rejects grouped, random-slope, temporal and non-Gaussian covariance structures. Its covariance workspace is bounded by the neighbor count, but exact neighbor discovery and the local factorizations are currently written in R. On the recorded moderate Linux diagnostic it used smaller retained fit objects but was slower than optimized dense linear algebra. It is therefore a memory-scalable statistical foundation, not a general speed claim or a recommended production budget. Chapter 20 gives the full derivation and the predeclared NNL study defines the evidence required before recommending a budget. In separate fixed-parameter Linux processes at 2,000 rows, neighbor 20 used 130,496 KiB peak resident memory versus 300,132 KiB for dense fitting; the neighbor route also completed 5,000 rows at 134,112 KiB. These single-machine diagnostics demonstrate the intended storage path, not general performance or statistical adequacy.

Joint draws and derived targets

`sp_predict_draws()` constructs the entire predictive covariance matrix and draws all requested locations together. Gaussian draws use the exact conditional Gaussian equations. Binomial and Poisson draws use the local Gaussian link distribution from the Laplace fit. On the response scale, a latent target produces probabilities or intensities, while an observation target produces Bernoulli outcomes or Poisson counts.

With `parameter_uncertainty = TRUE`, the function also draws fitted parameters from a local observed-Hessian Gaussian approximation and rebuilds the conditioning calculation for every draw. This option fails when curvature is unusable. For Gaussian fits, regression and log-covariance blocks are treated as asymptotically independent because their cross-covariance is not available.

For a large set of pointwise predictions, `sp_predict_chunked()` applies the same mean and marginal-variance equations to consecutive row chunks. It is exact for the returned pointwise quantities and reduces the largest training-by-grid matrix from (nm) to (nc)

entries for chunk size (c). It does not calculate covariance across chunks and must not replace joint draws for regional or simultaneous targets.

`sp_predict_draws_neighbor()` provides a different bounded-memory route for joint draws. It visits prediction rows in a declared order and conditions each row on at most the nearest requested number of predecessors. Every small conditional uses the same fitted predictive covariance equations as the exact method. Keeping every predecessor reproduces the exact Gaussian factorization; smaller neighborhoods are explicitly approximate and order-dependent. The result retains the neighbor graph and conditional variances instead of inventing a full covariance matrix. This reduces grid covariance storage but does not change the dense training fit; the separate Gaussian-only `sp_fit_neighbor()` route is required when training covariance storage is the constraint. Optional fitted-parameter uncertainty rebuilds the fitted state and all local conditionals for each draw; it is memory-bounded but potentially expensive. Input-row ordering is the default; deterministic maximin ordering spreads early sites across the coordinate domain with linear storage, and an explicit row permutation is also accepted.

When only grouped weighted summaries are needed, `sp_predict_target_neighbor()` accumulates each point's contribution as it is drawn and deletes that point's draw vector after its last use in the neighbor graph. It returns only target-level draws, sparse normalized allocation rows, the graph and the observed peak number of active point vectors. Polygon-centre and repeated fractional-area allocations are accepted. The active width can still approach the complete grid for a dense predecessor graph, so it is measured rather than promised.

`sp_neighbor_covariance()` is the deliberate full-matrix diagnostic. It recursively reconstructs the covariance implied by a neighbor graph and can compare it with the ordinary exact predictive covariance using Frobenius, maximum covariance and marginal-variance errors. It is for choosing and auditing budgets on manageable subsets, not for large-grid production.

`sp_grid()` creates regular point centres inside a projected polygon or the convex hull of projected point data. It does not guess a projection. Grid IDs are ordered consistently from top to bottom and left to right.

`sp_predict_target()` summarises joint draws with means, standard deviations, quantiles and optional exceedance probabilities. When weights and groups are supplied, it calculates the weighted target separately inside every draw. This retains spatial covariance in regional uncertainty. `sp_polygon_weights()` automatically assigns projected point or grid-centre predictions to polygon regions and can use a supplied population-like point weight. This is centre- point allocation. `sp_polygon_area_weights()` instead intersects polygon cells with non-overlapping target regions, retains exact areas and cell fractions, and can multiply area by a supplied cell density. Unmatched cells and empty regions follow explicit error/drop policies. `sp_target_geometry()` joins the unrounded summaries to `sf` geometry by exact identifiers or an explicit row order and supplies a base spatial display.

What the fitted methods return

`print(fit)`

Shows the model type, formula, sample size, distance units and covariance parameters. It labels estimation as exact Gaussian ML, Laplace, Monte Carlo ML, or fixed covariance.

`summary(fit)`

Shows:

- regression coefficient estimates;
- conditional coefficient standard errors;
- estimate divided by standard error, labelled as a z value;
- covariance parameters and fixed smoothness;
- approximate covariance-parameter standard errors from the observed profile likelihood Hessian and delta method;
- the optimiser convergence code;
- exact or approximated maximised log-likelihood;
- for MCML fits, importance ESS, largest normalized weight and log-likelihood Monte Carlo standard error;
- distance units.

The current summary does not provide boundary corrections or joint analytic propagation of covariance-parameter uncertainty into mean and prediction uncertainty. `confint(fit)` provides transformed local Wald intervals, and Hessian-based joint prediction draws are available separately. A non-positive-definite covariance Hessian produces missing standard errors rather than silently altered curvature.

Stable downstream tables

`sp_model_table(fit)` returns one unrounded row per mean or covariance parameter. It includes estimation status and the distance unit for the Matérn scale. Its coefficient p-values are large-sample Wald summaries; covariance parameters deliberately receive no automatic p-value because variance boundaries invalidate a routine normal interpretation.

`sp_prediction_table(draws, threshold = ..., probs = ...)` calculates the mean, sample standard deviation, requested type-8 quantiles and optional exceedance frequency from joint prediction draws. It also accepts an existing grouped target result. `sp_validation_table()` reshapes cross-validation, predictive assessment and parameter-recovery metrics to the common fields `assessment`, `scope`, `id`, `metric` and `value`. These are machine-readable contracts; publication formatting and maps remain downstream views.

`fitted(fit)` and `residuals(fit)`

For Gaussian models, `fitted(fit)` returns conditional spatial predictions at the observed locations. For binomial and Poisson models it returns conditional response means at the latent mode. `residuals(fit)` subtracts those conditional means from the observations.

`logLik(fit)`

Returns the maximised exact Gaussian log-likelihood, non-Gaussian Laplace approximation, or normalized importance-sampling likelihood after `sp_mcml()`. Its degrees of freedom count the mean coefficients and every estimated spatial, nugget, grouped, temporal and intervention parameter.

Matérn correlation as a standalone function

The direct call is

```
matern_cor(distance, scale, smoothness)
```

All distances must be numeric, finite, non-missing and non-negative. Scale and smoothness must each be one finite positive number. The function preserves the dimensions and names of a vector, matrix or array.

Examples:

```
matern_cor(c(0, 1, 2, 5), scale = 2, smoothness = 0.5)

locations <- cbind(east = c(0, 1, 2), north = c(0, 1, 0))
distance_matrix <- as.matrix(dist(locations))
correlation_matrix <- matern_cor(
  distance_matrix,
  scale = 1.5,
  smoothness = 0.5
)
```

Simulation, variograms and diagnostics

`simulate(fit, nsim = 20, seed = 41)` generates unconditional Gaussian responses at the fitted sites. Each column has the fitted regression mean and full fitted response covariance. It conditions on estimated parameters rather than propagating parameter uncertainty.

`sp_variogram(fit)` calculates

$$\hat{\gamma}_k = \frac{1}{n_k} \sum_{(i,j) \in P_k} \frac{1}{2}(e_i - e_j)^2$$

for distance-bin pair sets P_k. It returns the pair count n_k, empirical semivariance and, for Gaussian fits, fitted model semivariance. Binomial/Poisson fits accept Pearson, signed deviance or seeded randomized- quantile residuals and deliberately return no fitted response-scale curve. `plot(sp_variogram(fit))` displays the available empirical and fitted values.

`sp_diagnostics(fit)` returns mean-model residuals, conditional residuals and decorrelated residuals. If $V = LL^\top$, the latter are

$$L^{-1}(y - o - X\hat{\beta}).$$

They should be approximately independent standard normal values under an adequate fitted model, subject to parameter estimation. `plot()` supplies a residual-index panel and normal quantile plot.

Grouped intercepts and slopes

`random = ~ clinic + survey` adds independent zero-mean random intercepts for each observed level. Their variances may be fixed or estimated. Known levels contribute to conditional prediction; an unseen level is rejected unless the caller explicitly requests `unseen = "marginal"`, which retains its variance but gives it zero covariance with fitted groups.

A grouped slope is declared explicitly so its covariance assumption is visible:

```
grouped_age <- sp_random_slope(
  group = "clinic",
  slope = "age_centered",
  covariance = matrix(c(0.30, 0.05, 0.05, 0.08), 2, 2),
  estimate = TRUE
)

fit <- sp_fit(
  outcome ~ age_centered,
  data = survey_points,
  coords = ~ east_km + north_km,
  random_slope = grouped_age,
  control = sp_control(distance_units = "km")
)
```

Every clinic receives a correlated random intercept and age slope. With the default `estimate = FALSE`, the declared positive-definite matrix stays fixed, is labelled fixed in

model tables, has no invented standard error, and is not counted as an estimated degree of freedom. With `estimate = TRUE`, it is the starting value. SPmodels optimizes an unconstrained lower-triangular Cholesky factor with log-positive diagonal entries, so the reconstructed covariance is positive definite at every evaluation. Natural variances and covariance, delta-method standard errors and fitted degrees of freedom are then reported.

An estimated intercept-slope component requires at least three observed groups and at least two distinct slope values within each group. It may not be paired with a separate estimated random intercept for the same group, and two estimated slope components may not share one grouping variable. These are minimum checks; a slope-only component likewise needs at least two groups and two non-zero slope observations per group. They are not proof of strong identification; covariance inference can still be unstable near a zero-variance boundary. Lists of specifications add independent grouped components. Fitting, MCML, prediction, simulation, variograms, validation refits and recovery all use the same fitted covariance. Centering the slope covariate is strongly recommended because its zero defines the random intercept.

When a group-constrained validation fold deliberately removes a complete fitted grouping level, `sp_cross_validate(..., unseen = "marginal")` must be requested explicitly. The default error prevents an accidental change from conditional known-group validation to population-level new-group validation.

Non-Gaussian fitting and building blocks

`sp_fit()` accepts binomial-logit and Poisson-log families. It maximises a Laplace approximation to their marginal likelihood. This preliminary fit has the accuracy limitations of a local quadratic approximation; `sp_mcml()` is a separate, explicit correction rather than an unlabelled change of method.

Binomial data may be Bernoulli values, `cbind(successes, failures)`, or proportions with integer trial `weights`. Poisson data use integer counts; known exposure belongs in `offset(log(exposure))`. The latent covariance is a Matérn spatial component plus an independent latent component, both before the inverse link.

For every candidate regression and covariance parameter vector, the fitter finds the latent mode and evaluates the response likelihood, Gaussian prior quadratic, covariance determinant and curvature determinant. See equations L-01–L-16 in the technical derivations. The reported fitted values are conditional means at the latent mode, not marginal means integrated over latent uncertainty. Non-Gaussian `predict()` uses the local Gaussian posterior and integrates it through the log or logit link. It reports link- or response- scale uncertainty conditional on fitted parameters. Non-Gaussian `simulate()` draws a latent Gaussian vector and then success counts or Poisson counts from the fitted response family. `sp_diagnostics()` returns response, Pearson, signed deviance and locally seeded randomized quantile residuals at the latent mode. The randomized residual draws uniformly between the fitted CDF below and at the observed count, then applies a standard-normal quantile. These conditional diagnostics are not claimed to be spatially decorrelated.

`matern_scale_derivative(distance, scale, smoothness, order)` evaluates the first or second derivative of Matérn correlation with respect to scale. The implementation uses analytic Bessel identities and is checked against both the exponential special case and centred finite differences.

`sp_latent_mode()` accepts outcomes, a fixed linear predictor and a supplied latent covariance matrix. For binomial-logit or Poisson-log observations it maximises

$$h(s) = \sum_i \ell_i(a_i + s_i) - \frac{1}{2} s^\top \Sigma^{-1} s.$$

It returns the unique mode, final score, iteration status, conditional means and inverse local curvature. A damped Newton method prevents an accepted step from reducing the objective. This supports the Laplace fitter and proposal construction; the mode operation by itself is not MCML or a fitted model.

`sp_latent_sample()` uses that local curvature to precondition a Metropolis-adjusted Langevin proposal. It applies the full asymmetric proposal correction and returns retained draws plus the realised acceptance rate. `sp_mcmc_control()` makes chain count, dispersed-start scale, iterations, burn-in, thinning and step size explicit. Multiple chains retain their own draw arrays and acceptance rates. These are conditional latent draws for supplied parameters; regression and covariance parameters are not estimated by this call.

`sp_sampler_diagnostics()` summarises each retained latent component by its mean, standard deviation, autocorrelation-aware effective sample size and Monte Carlo standard error. With multiple chains it reports the maximum of rank-normalized split and folded rank-normalized split R-hat. Chains may be selected explicitly for diagnosis. For one selected component it returns per-chain traces and average autocorrelation values and has a two-panel plot method. These diagnostics are necessary checks, not proof of convergence.

Monte Carlo likelihood correction

`sp_mcml(fit)` starts from a non-Gaussian Laplace fit. It constructs a normalized multivariate Gaussian importance distribution from the fitted latent mode and local curvature, draws latent vectors independently, and maximizes the resulting Monte Carlo estimate of the marginal likelihood. The same draws are used at every optimizer evaluation, so the objective is deterministic within a call. A supplied seed is local and reproducible.

Supplying several distinct `proposal_scale` values constructs one normalized same-centre mixture. Draws are balanced across scales and every weight divides by the complete mixture density. This combines narrow and wide components without discarding the likelihood normalizing constant. Its likelihood MCSE uses the component-stratified variance formula.

With `adapt = TRUE`, a separate pilot sample is weighted at the preliminary fit. Its weighted centre and covariance are shrunk toward the original Laplace moments, then a new main sample is drawn from the resulting normalized proposal. The pilot does not

enter the likelihood estimate. Conditional on that pilot, ordinary importance-sampling normalization therefore targets the same marginal likelihood. The fit retains the complete pilot and both sets of moments. Pilot ESS is checked because a dominant pilot weight can make the adapted moments unstable. A pilot below `ess_warning` triggers an explicit, recorded fallback to the original moments; lowering the threshold is how a user deliberately overrides that gate. Adaptation is an auditable option, not an automatic guarantee of improvement.

The returned fit records all latent proposal draws and final log weights. Its `mcml` component reports the importance effective sample size and fraction, largest normalized weight, and a delta-method Monte Carlo standard error for the log likelihood. A warning is issued below the declared ESS threshold. These diagnostics are essential: increasing the number of draws cannot repair a proposal that misses an important region.

The reported MCML likelihood is absolute because the proposal density is normalized; no unknown reference likelihood is dropped. Prediction is then reconstructed at the MCML estimates, but the conditional latent prediction step remains a local Gaussian approximation. Cross-validation of an MCML fit repeats the correction inside each training fold.

`sp_mcml_replicates()` reruns the whole correction with independent draws. It retains every fit and reports the mean, between-replicate standard deviation and range of each optimized parameter. It also retains empirical parameter-level Monte Carlo covariance, average fitted-Hessian sampling covariance and their explicit sum. These components remain separately named: Monte Carlo sensitivity is not silently relabelled as scientific sampling uncertainty, and unusable Hessians leave sampling and combined summaries missing.

When adaptation is enabled, replicate and precision histories include pilot ESS fraction, dominant weight, relative MCSE and the used/fallback decision. This makes a rejected pilot visible without opening each retained fit.

`sp_mcml_precision()` starts at the control object's sample size and increases it by a declared geometric factor. It stops only when both the relative likelihood MCSE and ESS-fraction targets pass, or after evaluating the exact maximum sample size. Each attempt is recorded. Optional proposal refresh uses the preceding fitted latent mode and curvature for the next attempt. Failure at the cap produces a warning and is never re-labelled as success.

Fixed decay-adjusted intervention histories

`sp_intervention()` normalizes a unit-by-time intervention history and records an immediate proportional reduction `alpha` and positive decay time. For a survey at time (t), each delivered intervention at (u<t) multiplies baseline prevalence by

$$1 - \alpha \exp\{-(t-u)/\text{decay}\}.$$

The row multiplier is the product over all earlier interventions in the same unit. An empty history gives one. An intervention recorded at exactly the survey time is excluded, which makes the timing convention explicit: survey first, rollout second. `sp_intervention_multiplier()` exposes these row values directly for auditing.

For a binomial `sp_fit()`, observed prevalence is the baseline logistic prevalence times this multiplier. The adjusted likelihood and its derived latent score/curvature are used during Laplace fitting. The same history is retained through MCML correction, response prediction, fitted simulation, cross-validation refits and model tables. Prediction rows must contain the declared unit and time columns.

`alpha` and decay may instead be estimated jointly by the labelled Laplace route, using logit and log parameterizations. An optional normalized beta penalty for `alpha` is included in the recorded criterion. Observed-Hessian uncertainty is transformed to the ordinary scale, and `sp_intervention_curve()` uses it for pointwise delta-method bounds. MCML can re-estimate both parameters on their constrained scales, includes the optional normalized alpha penalty and transforms its curvature back to ordinary units. Residual temporal covariance can be combined with the intervention history. Chapter 14 gives the intervention equations and validation map.

Validation completed so far

The current implementation has local, independent checks for:

- Matérn correlation equal to one at zero distance;
- exponential and other half-integer Matérn identities;
- invariance when distance and scale are changed by the same factor;
- positive-semidefinite correlation matrices, within floating-point tolerance;
- direct generalised least-squares coefficients;
- direct Gaussian log-likelihood evaluation;
- direct Gaussian conditional prediction means;
- latent and future-observation prediction variances;
- invariance to permutation of data rows;
- positive fitted covariance parameters and optimiser completion;
- covariance-Hessian dimensions and summary reporting;
- explicit rejection of unsupported or invalid inputs;
- exact seeded Gaussian simulation through the stored Cholesky factor;
- variogram pair accounting and fitted semivariance values;
- direct reconstruction of decorrelated residuals;
- projected `sf` fitting and prediction with rejection of geographic or mismatched CRSs;
- direct reconstruction of the non-Gaussian Laplace log likelihood;
- equivalent binomial count and proportion encodings;
- binomial row-permutation invariance and Poisson offset handling;
- direct Poisson lognormal prediction moments and binomial logistic-normal numerical integration;

- conditional MALA posterior moments against one-dimensional numerical integration and basic sampler-diagnostic bounds;
- MCML importance likelihood against independent one-dimensional numerical integration, exact seeded reproducibility, weight diagnostics, binomial and Poisson operation, downstream prediction and fold-wise refitting;
- exact seeded non-Gaussian latent/response simulation transforms and direct Pearson/deviance residual definitions;
- geometry-driven folds with repeated-location protection, canonical labels and direct cross-validation use;
- repeated Gaussian and binomial simulation/refitting with an independently reconstructed first replicate and direct bias/RMSE summaries;
- fixed-fold predictions reproduced by an independent manual refit;
- empirical CRPS reproduced by brute-force pairwise draw differences;
- source build, clean installation, examples and `R CMD check`.

These checks establish consistency with the stated equations for small, well-conditioned examples. They do not establish broad scientific validity, large-data performance or numerical agreement with the separately managed reference environment. That comparison has not yet been run.

What is not implemented

The following are not available and must not be implied by the presence of the `sp_fit()` name:

- non-Gaussian links other than binomial logit and Poisson log;
- random-slope structures beyond one numeric slope per component, and boundary-corrected inference for their covariance matrices (fixed or constrained-estimated slope and intercept-slope matrices are implemented);
- estimation of Matérn smoothness;
- automatic CRS projection or transformation;
- anisotropic or non-stationary covariance;
- non-separable, anisotropic or non-stationary space-time dependence;
- boundary-corrected inference for weakly identified temporal scales;
- exact scalable joint prediction or an exact scalable replacement for dense fitting (exact chunked pointwise prediction, approximate bounded-memory joint draws and an opt-in approximate Gaussian training likelihood are implemented; the training route still requires its predeclared recovery, budget and cross-platform studies);
- further domain-specific fold constructors without a declared consumer need (whole-group constraints and arbitrary supplied identifiers are accepted);
- automatic cartographic styling, interactive maps or consumer-specific map adapters;
- consumer-specific polygon target definitions;
- journal-specific typesetting and HTML/office report export;

- likelihood-profile or boundary-corrected confidence intervals and analytic covariance-parameter propagation in prediction intervals; transformed local Wald intervals and optional Hessian-based predictive parameter draws are available;
- large-data approximations or compiled numerical kernels.

Can the package currently replace a broad spatial workflow?

No. The current implementation covers Gaussian fitting, simulation and diagnostics; Laplace binomial/Poisson fitting; point and joint prediction; projected grids and exact fractional-area regional targets; fixed-fold, draw-based and calibration assessment; importance-sampling MCML correction; and fitted decay-adjusted intervention histories. The broader programme still includes unsupported model classes, large-data methods, richer geometry, reporting and downstream-workflow validation.

The intended outcome is independent capability coverage with an SPmodels-owned interface, not a copy of another package's function catalogue or internal implementation. Completion can be claimed only when each supported capability has its own specification, derivation or algorithm, tests, validation evidence and consumer checks.

Next development steps

The next work should proceed in dependency order.

Stage 1: complete validation of the implemented foundation

1. Measure externally observed peak RSS by NNL configuration and on additional supported platforms. Per-fit timings and the full-run peak RSS are recorded alongside archived statistical-accuracy and stability results in the NNL benchmark report. GRS and TSC recovery studies are now recorded; run the remaining registered MCP proposal-policy and NNP prediction studies.
2. The GRS spatial-range diagnostic found broad Hessian uncertainty, poor local interval coverage and estimated scales far from truth. The TSC cases reached their 400-attempt caps but were inconclusive: usable-fit rates were below 90%, temporal-scale bias-MCSEs were far above target, and few MCML response sets passed all repeat gates. See the case-specific reports; neither study establishes recovery or reference agreement.
3. Extend transformed local confidence intervals with likelihood profiling or justified boundary corrections, and add analytic covariance-parameter propagation to prediction intervals where reliable.

4. Retain the checked lsda binomial example and add count/grid-target examples only when the data support and coordinate alignment are valid; the current suitability audit records why the inspected areal/rate/grid objects are not being forced into point workflows.
5. Run the pinned reference-validation cases and record discrepancies.

Stage 2: non-Gaussian geostatistical models

1. Extend the implemented response conventions only where a scientific use case requires it.
2. The registered MCML proposal study found that none of the tested policies was operational on either synthetic fixture; the authorized 197-site Loa loa diagnostic also had severe weight degeneracy. Redesign proposals from the retained ESS/weight/MCSE evidence and predeclare a new independent study before selecting settings or making reliability claims.
3. Integrate the implemented precision targets and repeated-chain checks into fitted stochastic workflows where scientifically required.
4. Extend fitted non-Gaussian diagnostics beyond the implemented empirical variograms and permutation/full-refit envelopes where a formal scientific use case identifies another required model check.
5. Validate deterministic components, stochastic equivalence and uncertainty.

Stage 3: reusable spatial workflows

1. Extend geometry-driven, environmental, regular-block, whole-group and buffered folds only when another named consumer constraint is required.
2. Extend named multi-scenario simulation/refit recovery, if needed, with shared multi-scenario precision targets; consider use the implemented replicate- or scenario-level parallel routes after profiling identifies the appropriate granularity.
3. Extend grid and boundary construction with consumer-specific schemas.
4. Add consumer-specific regional target definitions where authorized.
5. Evaluate the implemented bounded-memory nearest-predecessor joint draws across defensible prediction orders and neighbor budgets; exact bounded-memory pointwise prediction remains available through `sp_predict_chunked()`. The NNL accuracy study found no general training budget; per-configuration memory and cross-platform timing remain open. Keep recommendations scenario-specific and optimize its R implementation only from broader measured evidence.
6. Extend the current target and diagnostic displays with reviewed publication/cartographic styling.

Stage 4: extended model structures

1. Evaluate the constrained random-slope covariance estimator in the pinned validation designs and add boundary-corrected inference where scientifically justified; marginal unseen-level prediction is already explicit.
2. Validate the implemented non-Gaussian temporal-scale estimator in the pinned cases and study scientifically justified non-separable models.
3. Extend fitted intervention models with consumer-reviewed decision summaries and study richer temporal dependence where scientifically identifiable.
4. Extend the implemented matched prediction updates into richer authorized consumer-facing workflows.

Stage 5: adoption and release

1. Run MBGapp integration tests without the reference package installed.
2. Add lsda and course examples while avoiding a circular runtime dependency.
3. Profile realistic workloads and optimise only verified bottlenecks.
4. Complete package identity, licence, citation, CRAN and optional R-universe decisions.

Current conclusion

SPmodels now supports a transparent formula-to-fit-to-predict Gaussian spatial workflow and fitted binomial/Poisson models through a labelled Laplace approximation with optional importance-sampling MCML correction, including point prediction integrated through nonlinear links. Separable residual space-time dependence is carried through fitted workflows, with temporal-scale estimation for Gaussian models. Independent MCML repeats and capped precision-directed escalation are implemented. Fixed or constrained-estimated random slopes and correlated grouped intercept-slope effects are also implemented. It is still an incomplete package: richer space-time models, large-data methods and recorded reference validation remain major milestones.